Introduction

Abstract: This project analyzes tennis match data to predict the winner based on player ranks, points, odds, and other match features.

Student Name: Markiian Ivaniv

Imports

```
# Basic libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# AutoViz
!pip install autoviz
from autoviz.AutoViz_Class import AutoViz_Class

# PyCaret
!pip install pycaret
from pycaret.classification import *

# Warnings
import warnings
warnings.filterwarnings('ignore')
```

Show hidden output

Input Data

```
url = "https://raw.githubusercontent.com/mivaniv23/Data-Mining-Project-2025/refs/
df = pd.read_csv(url)
df.head()
```

Show hidden output

Explore Data

```
# Summary statistics
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64951 entries, 0 to 64950
Data columns (total 17 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Tournament  64951 non-null  object
 1   Date        64951 non-null  object
 2   Series      64951 non-null  object
 3   Court       64951 non-null  object
 4   Surface     64951 non-null  object
 5   Round       64951 non-null  object
 6   Best of     64951 non-null  int64
 7   Player_1    64951 non-null  object
 8   Player_2    64951 non-null  object
 9   Winner      64951 non-null  object
 10  Rank_1      64951 non-null  int64
 11  Rank_2      64951 non-null  int64
 12  Pts_1       64951 non-null  int64
 13  Pts_2       64951 non-null  int64
 14  Odd_1       64951 non-null  float64
 15  Odd_2       64951 non-null  float64
 16  Score       64951 non-null  object
dtypes: float64(2), int64(5), object(10)
memory usage: 8.4+ MB
```

```
# Statistical summary
df.describe()
```

Show hidden output

```
# Analysis of Nulls
df.isnull().sum()
```

Show hidden output

```
# AutoViz visualization
AV = AutoViz_Class()
auto_viz = AV.AutoViz(url)
```

Show hidden output

Prepare Data

```
# Drop columns with many missing values or irrelevant
df_clean = df.drop(columns=['Tournament', 'Date', 'Series', 'Court', 'Surface', '
# Fill missing values
df_clean['Rank_1'] = df_clean['Rank_1'].fillna(df_clean['Rank_1'].median())
df_clean['Rank_2'] = df_clean['Rank_2'].fillna(df_clean['Rank_2'].median())
df_clean['Pts_1'] = df_clean['Pts_1'].fillna(df_clean['Pts_1'].median())
df_clean['Pts_2'] = df_clean['Pts_2'].fillna(df_clean['Pts_2'].median())
df_clean.dropna(inplace=True)
df_clean.head()
```

	Player_1	Player_2	Winner	Rank_1	Rank_2	Pts_1	Pts_2	Odd_1	Odd_2
0	Dosedel S.	Ljubicic I.	Dosedel S.	63	77	-1	-1	-1.0	-1.0
1	Clement A.	Enqvist T.	Enqvist T.	56	5	-1	-1	-1.0	-1.0
2	Escude N.	Baccanello P.	Escude N.	40	655	-1	-1	-1.0	-1.0
3	Knippschild J.	Federer R.	Federer R.	87	65	-1	-1	-1.0	-1.0

```
df_clean['Winner'].value_counts()
```

Winner	count
Federer R.	1151
Djokovic N.	1032
Nadal R.	1007
Ferrer D.	677
Murray A.	670
...	...
Mecir M.	1
Langer N.	1
Trifu G.	1
Tebbutt M.	1
Collignon R.	1

1149 rows × 1 columns

dtype: int64

```
# Count the occurrences of each winner
winner_counts = df_clean['Winner'].value_counts()

# Keep only top 20 most frequent winners
top_winners = df_clean['Winner'].value_counts().nlargest(20).index
df_top = df_clean[df_clean['Winner'].isin(top_winners)]

# Check the filtered data
df_top['Winner'].value_counts()
```

Winner	count
Federer R.	1151
Djokovic N.	1032
Nadal R.	1007
Ferrer D.	677
Murray A.	670
Berdych T.	575
Gasquet R.	573
Roddick A.	564
Monfils G.	549
Wawrinka S.	533
Cilic M.	531
Verdasco F.	524
Robredo T.	502
Hewitt L.	494
Simon G.	477
Lopez F.	471
Isner J.	465
Davydenko N.	458
Youzhny M.	456
Dimitrov G.	439

dtype: int64

Model

```
from pycaret.classification import *
```

```
clf1 = setup(data=df_top, target='Winner', session_id=123,)

# Compare models
top_models = compare_models(include=['lr', 'dt', 'rf', 'nb'], n_select=3)
```

	Description	Value
0	Session id	123
1	Target	Winner
2	Target type	Multiclass
3	Target mapping	Berdych T.: 0, Cilic M.: 1, Davydenko N.: 2, Dimitrov G.: 3, Djokovic N.: 4, Federer R.: 5, Ferrer D.: 6, Gasquet R.: 7, Hewitt L.: 8, Isner J.: 9, Lopez F.: 10, Monfils G.: 11, Murray A.: 12, Nadal R.: 13, Robredo T.: 14, Roddick A.: 15, Simon G.: 16, Verdasco F.: 17, Wawrinka S.: 18, Youzhny M.: 19
4	Original data shape	(12148, 9)
5	Transformed data shape	(12148, 9)
6	Transformed train set shape	(8503, 9)
7	Transformed test set shape	(3645, 9)
8	Numeric features	6
9	Categorical features	2
10	Preprocess	True
11	Imputation type	simple
12	Numeric imputation	mean
13	Categorical imputation	mode
14	Maximum one-hot encoding	25

15	Encoding method	None
16	Fold Generator	StratifiedKFold
17	Fold Number	10
18	CPU Jobs	-1
19	Use GPU	False
20	Log Experiment	False
21	Experiment Name	clf-default-name
22	USI	571f

Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)

Evaluate

```
# Create and evaluate a Random Forest model
rf_model = create_model('rf')
evaluate_model(rf_model)
predict_model(rf_model)
```

Fold	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.9154	0.9907	0.9154	0.9163	0.9143	0.9103	0.9105
1	0.9107	0.9908	0.9107	0.9138	0.9102	0.9054	0.9056
2	0.9177	0.9919	0.9177	0.9189	0.9175	0.9129	0.9130
3	0.9294	0.9938	0.9294	0.9329	0.9296	0.9253	0.9254
4	0.9200	0.9935	0.9200	0.9228	0.9201	0.9153	0.9154
5	0.9141	0.9920	0.9141	0.9150	0.9133	0.9090	0.9092
6	0.9188	0.9949	0.9188	0.9207	0.9188	0.9140	0.9141
7	0.9035	0.9915	0.9035	0.9067	0.9035	0.8978	0.8979

8	0.9212	0.9958	0.9212	0.9241	0.9212	0.9165	0.9167
9	0.9294	0.9934	0.9294	0.9311	0.9290	0.9252	0.9253
Mean	0.9180	0.9928	0.9180	0.9202	0.9178	0.9132	0.9133
Std	0.0075	0.0016	0.0075	0.0075	0.0076	0.0079	0.0079

Plot Type:

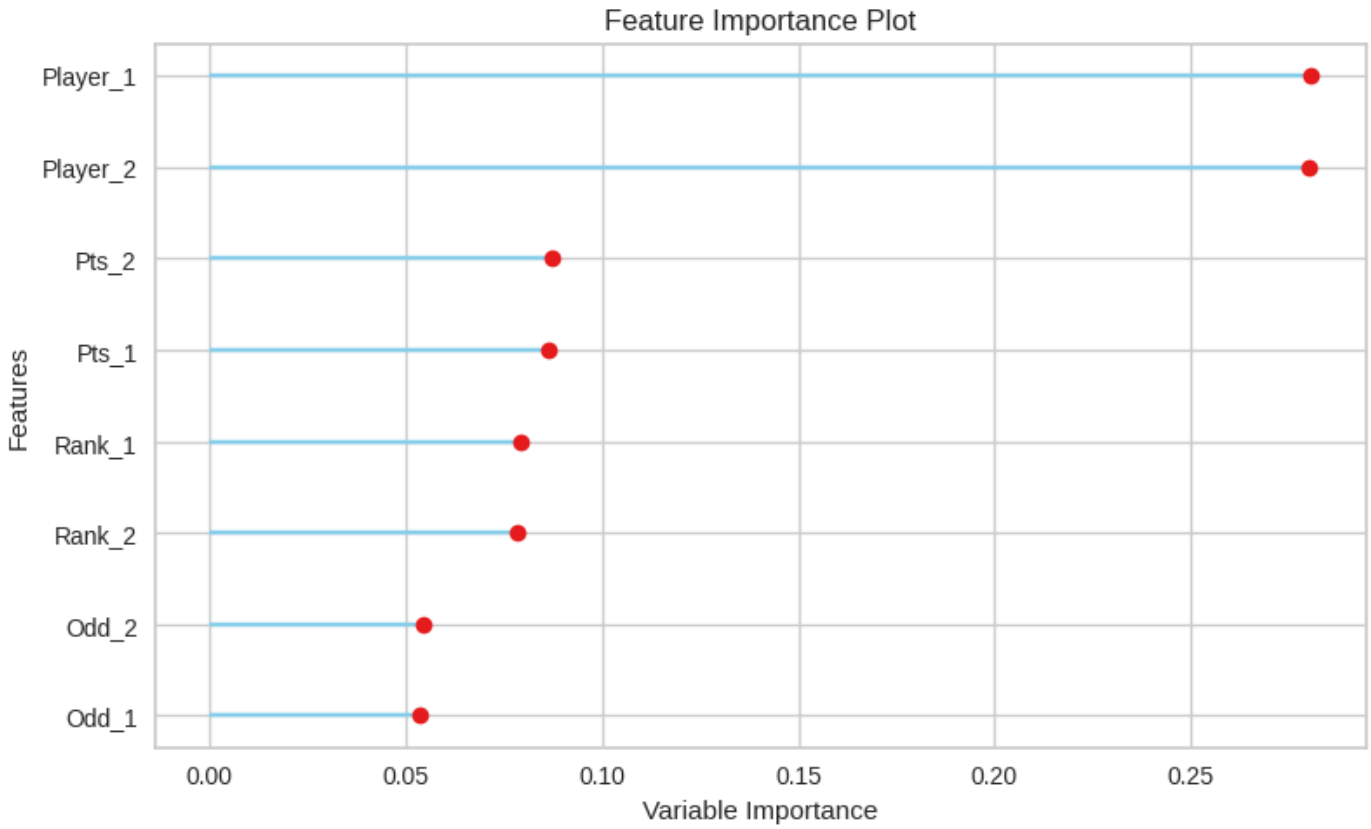
Pipeline Plot	Hyperparameters	AUC	Confusion Matrix
Threshold	Precision Recall	Prediction Error	Class Report
Feature Selection	Learning Curve	Manifold Learning	Calibration Curve
Validation Curve	Dimensions	Feature Importance	Feature Importanc...
Decision Boundary	Lift Chart	Gain Chart	Decision Tree
KS Statistic Plot			

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	Random Forest Classifier	0.9309	0.9937	0.9309	0.9314	0.9306	0.9268	0.9268

	Player_1	Player_2	Rank_1	Rank_2	Pts_1	Pts_2	Odd_1	Odd_2	Winner
15171	Beck K.	Youzhny M.	41	24	-1	-1	2.62	1.44	Youzhny M.
14284	Hewitt L.	Chela J.I.	3	24	-1	-1	1.12	5.50	Hewitt L.
38069	Murray A.	Cilic M.	6	37	4795	1085	1.50	2.50	Cilic M.
61987	Van De Zandschulp B.	Monfils G.	70	68	810	837	2.20	1.67	Monfils G.
57538	Gasquet R.	Majchrzak K.	75	81	808	762	1.72	2.10	Gasquet R.
...	...	...	...	...	...	...	...	...	...
32898	Djokovic N.	Hewitt L.	1	181	13630	265	1.01	21.00	Djokovic N.
60009	Ivashka I.	Wawrinka S.	73	84	741	717	2.63	1.50	Wawrinka S.

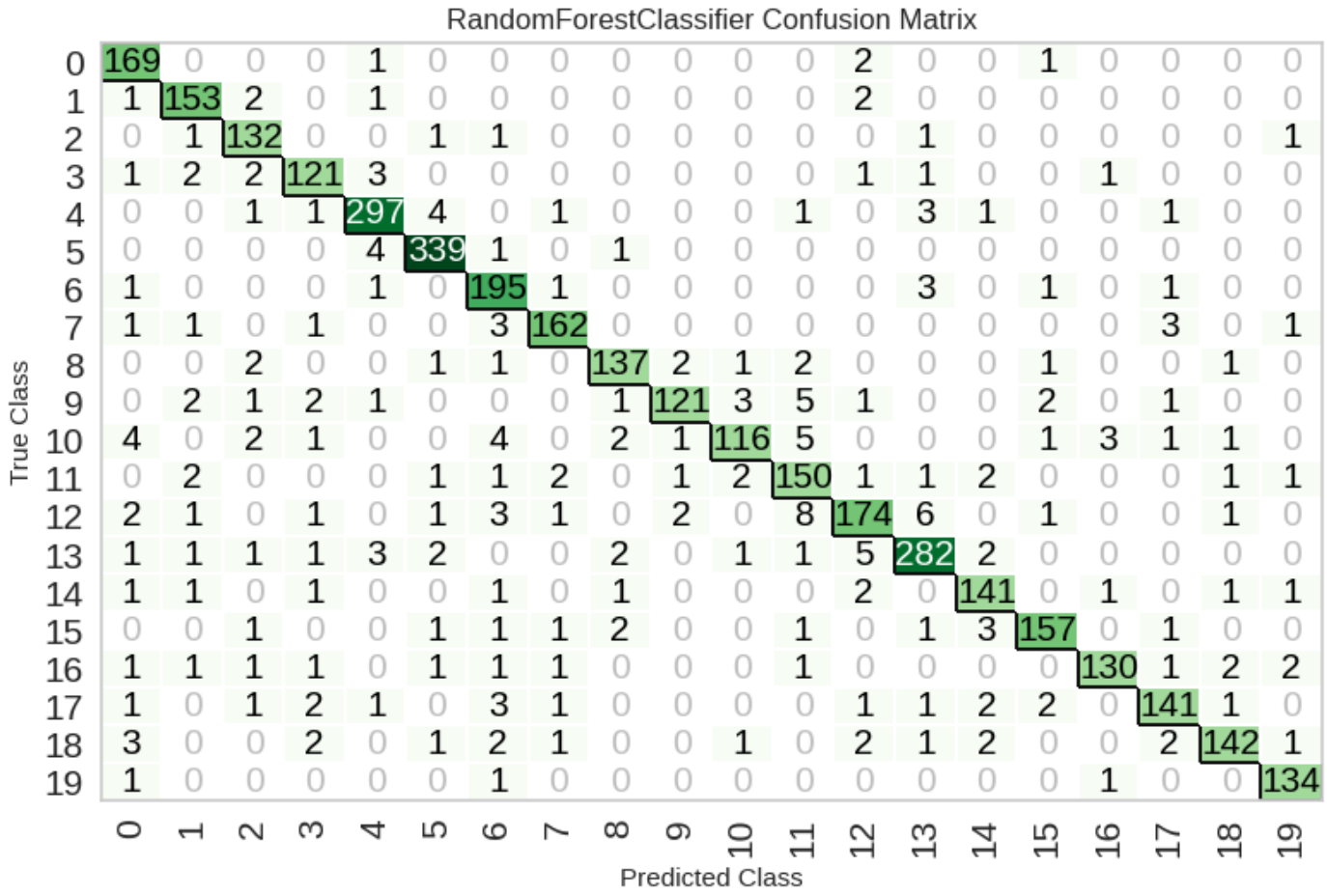
Feature Importance

```
plot_model(rf_model, plot='feature')
```



Confusion Matrix

```
plot_model(rf_model, plot='confusion_matrix')
```



Tune, Best Model

```
tuned_rf = tune_model(rf_model)
final_rf = finalize_model(tuned_rf)
save_model(final_rf, 'final_rf_model')
```

Fold	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.7720	0.9748	0.7720	0.8014	0.7732	0.7590	0.7604

1	0.7415	0.9726	0.7415	0.7588	0.7398	0.7266	0.7276
2	0.7556	0.9753	0.7556	0.7807	0.7534	0.7415	0.7428
3	0.7682	0.9742	0.7682	0.8051	0.7701	0.7550	0.7567
4	0.7765	0.9745	0.7765	0.8111	0.7711	0.7638	0.7655
5	0.7588	0.9735	0.7588	0.7771	0.7561	0.7449	0.7461
6	0.7624	0.9738	0.7624	0.7900	0.7568	0.7487	0.7503
7	0.7729	0.9702	0.7729	0.7955	0.7740	0.7600	0.7611
8	0.7706	0.9764	0.7706	0.8071	0.7685	0.7575	0.7593
9	0.7682	0.9740	0.7682	0.7802	0.7651	0.7548	0.7558
Mean	0.7647	0.9739	0.7647	0.7907	0.7628	0.7512	0.7526
Std	0.0099	0.0016	0.0099	0.0156	0.0104	0.0105	0.0107

```
Fitting 10 folds for each of 10 candidates, totalling 100 fits
Original model was better than the tuned model, hence it will be returned. NOT
Transformation Pipeline and Model Successfully Saved
(Pipeline(memory=Memory(location=None),
          steps=[('label_encoding',
                  TransformerWrapperWithInverse(exclude=None, include=None,
                                                transformer=LabelEncoder())),
                 ('numerical_imputer',
                  TransformerWrapper(exclude=None,
                                     include=['Rank_1', 'Rank_2', 'Pts_1',
                                              'Pts_2', 'Odd_1', 'Odd_2'],

transformer=SimpleImputer(add_indicator=False,
                                                                copy=True,

fill_value=None,

keep_empty_featur...
                  RandomForestClassifier(bootstrap=True, ccp_alpha=0.0,
                                         class_weight=None, criterion='gini',
                                         max_depth=None, max_features='sqrt',
                                         max_leaf_nodes=None,
max_samples=None,
                                         min_impurity_decrease=0.0,
                                         min_samples_leaf=1,
min_samples_split=2,
                                         min_weight_fraction_leaf=0.0,
                                         monotonic_cst=None,
```

```
n_estimators=100
```

References

- Tennis dataset from Kaggle: https://www.kaggle.com/datasets/dissfya/atp-tennis-2000-2023daily-pull
- PyCaret Documentation: https://pycaret.gitbook.io/docs/
- AutoViz GitHub: https://github.com/AutoViML/AutoViz